Process Report – StudyHelper

Institution:
VIA University College

Course & Semester:
SEP4, 4th Semester

Authors (Student Numbers):
Alexandru Savin(354790)
Cristina Aurelia Matei (354776)
Damian Michal Choina (354789)
Eduard Fekete (355323)
Jakub Maciej Baczek (354814)
Karina Rubahova (354565)
Mara-Ioana Statie (354536)
Marta Zrno (355351)
Piotr Junosz (355502)
Tymoteusz Krzysztof Zydkiewicz (355413)

Supervisors:
Erland Ketil Larsen
Joseph Chukwudi Okika
Kasper Knop Rasmussen
Markéta Tranberg

Date:
May 25, 2026

Character Count:
26,966 characters

TABLE OF CONTENTS

1. INTRODUCTION

Provide a factual, chronological overview of the project from kick-off to submission, grounded in concrete sources (logbook entries, meeting minutes, sprint retrospectives). Highlight major milestones and any significant pivots in direction.]

We chose the project StudyHelper probably because of the relevance of the topic in all our lives. The idea of a device that can help navigate in choosing the best environment for studying was at the intersection of our interests and feasibility within the project scope.

The initial phase of the project involved heavy brainstorming sessions and discussions. The main point of these debates was refining our problem statement to ensure it would be meaningful to solve with a solution that would be technically relevant for all sub-teams in a way that satisfies the overall expectations of the project.

2. GROUP WORK

2.1 Group Composition and Profiles

[Introduce each group member briefly — background, relevant skills, and prior experience. If the group is multicultural or interdisciplinary, highlight this and reflect on how it influenced the collaboration.]

Member	Background	Key Strengths	Role(s) in Project
[Name]	[Study/origin]	[Skills]	[e.g. Backend, PM]

2.2 Roles and Contributions

[Describe clearly how responsibilities were divided. Who led which areas? Was the division planned or did it emerge organically? Provide specific examples: “X took ownership of the database layer and led the sprint planning sessions in weeks 3–6.”]

2.3 Team Dynamics and Collaboration

[How did the team communicate day-to-day? What tools did you use (Discord, Jira, etc.)? Were there informal leadership dynamics? How were decisions made — by consensus, by vote, or by domain ownership?]

2.4 Conflict and Resolution

[Did any disagreements or tensions arise? How were they addressed? What did the team learn from handling conflict? If no significant conflict occurred, briefly note how the group maintained alignment.]

2.5 Social Loafing and Accountability

[Were there any instances where workload felt unevenly distributed? How did the group ensure accountability? What mechanisms (standups, task boards, peer review) helped keep everyone engaged?]

3. PROJECT INITIATION

3.1 Choosing the Problem Domain

[Why did your group choose this particular topic? Was it driven by personal interest, practical relevance, available resources, or suggestions from a supervisor/company? In hindsight, was this a good choice? What would you do differently?]

3.2 Problem Statement Development

[How did you arrive at the final problem statement? Describe the iterations. Was the initial framing too broad, too narrow, or off-target? What clarified your thinking — literature, supervisor input, prototyping?]

3.3 Time Planning and Scheduling

[How did you plan the project timeline? Did you use sprints, Gantt charts, or a looser milestone structure? How realistic was the initial plan? Reflect on deviations: what caused delays, and how were they managed?]

3.4 Ethical Perspectives

[What ethical questions arise from the problem your project addresses? Consider: data privacy, environmental impact, fairness, accessibility, or societal effects. Were these considerations built into the project from the start, or addressed reactively?]

4. PROJECT EXECUTION

This section describes the development process, divided into our collective efforts and team-specific contributions.

4.1 Together Process

This section describes the development process, divided into our collective efforts and team-specific contributions.

4.1 Together Process

[Describe how the whole group worked together during the execution phase. How were the components (IoT, ML, Frontend) integrated? How did we handle cross-team dependencies and API contracts? Reflect on the effectiveness of our shared git workflow and communication.]

4.2 IOT Team Process

The IoT sub-team consisted of three members: Jakub Maciej Baczek, Damian Michal Choina, and Tymoteusz Krzysztof Żydkiewicz. None of the team members had prior experience with embedded systems programming in C, which meant the first sprint was spent largely on getting the environment working rather than writing application code. Setting up PlatformIO, understanding the AVR toolchain, and getting the first successful flash onto the Arduino took more time than anticipated, but it was a necessary foundation that paid off once development picked up pace.

4.2.1 Work Division

The team divided work based on voluntary preference rather than top-down assignment. At the start of each sprint, everyone stated what they were willing to pick up, and tasks were distributed accordingly. From previous SEP projects together the team had found that this approach works better than assigning tasks externally, since people are more invested in work they have chosen themselves. In practice, responsibilities emerged naturally: server communication and HTTP layer, sensor integration, main loop logic and testing - each had a clear owner without formal negotiation. The team maintained its

own backlog rather than using a shared tool like Trello or Jira, which kept things lightweight for a three-person team.

4.2.2 Development Milestones

The project ran across seven sprints and progress was measured against three concrete milestones rather than abstract story points. The first was getting the Arduino communicating with the server at all — once that worked end-to-end, even with hardcoded values, the rest of the work had a clear foundation to build on. The second milestone was the running session feature, where the device could start a session, send periodic data with keepalive pulses and end the session cleanly via button press. The third was the instant measurement feature, where a button press triggered a one-off sensor reading and sent it for immediate ML prediction. Reaching each of these felt like a genuine shift in confidence for the team.

4.2.3 Testing and CI/CD

One deliberate process decision was investing in automated testing and a CI/CD pipeline early in the project. Because the firmware runs on hardware, the team chose to run unit tests on the host machine rather than the device, using the Unity testing framework with FFF-generated fakes to replace AVR hardware dependencies. This allowed testing of application logic — server communication, session state, response parsing — without needing the physical Arduino present. A GitHub Actions workflow was set up to compile the firmware, run the full test suite, and upload coverage reports on every pull request. In practice the pipeline needed some adjustment as the project evolved, and having it fully configured from the very beginning would have saved some rework. That said, having it in place at all caught several issues before they reached the hardware and gave the team confidence during integration.

4.2.4 Integration with Other Teams

The API contract between the IoT team and the rest of the group was discussed at the start of the project, which gave a clear target for the communication layer. However, the contract evolved over time as certain features turned out to be impractical and requirements became clearer on both sides. Endpoint paths and response field names changed at points during development, which required adjustments to the firmware. These were manageable but reinforced the value of treating the API contract as a living document that all teams need to stay aligned on throughout the project, not just at the kickoff.

4.3 MAL Team Process

The Machine Learning and API (MAL) development followed an iterative path from data exploration to pipeline implementation. The following notes capture the key decisions and observations made during this process.

4.3.1 Mock Data Search and Goal Refinement

Initially, the focus was on how environmental noise influences focus. We attempted to combine datasets of focus-related noise effects with labeled sound categories from WAV files, extracting frequencies and loudness. However, we realized that "focus" cannot be measured directly. Consequently, we narrowed the goal to predicting a user-provided **Study Suitability Rating**, which made the target variable more grounded in user feedback.

4.3.2 Data Quality and Correlation Analysis

Further investigation led to the elimination of several datasets that appeared synthetic. We observed that in some candidate datasets, the distributions of features like humidity, noise, and light were suspiciously uniform, suggesting algorithmic generation rather than real sensor collection.

We also analyzed feature correlations as part of our validation process. Healthy datasets showed natural physical correlations (e.g., CO2, temperature, and humidity), while suspicious datasets exhibited either zero correlation or extreme overfitting potential. We decided to merge diverse datasets to create a more robust mock dataset.

4.3.3 Advanced Imputation Strategy

To handle missing values after merging, we implemented a sophisticated approach using the **MICE (Multivariate Imputation by Chained Equations)** framework. We enhanced this by:

- **Clustering**: Using k-means to find environment types (e.g., sessions made with high sun exposure vs. sessions made in labs).
- **ExtraTrees Estimator**: Modeling complex non-linear relationships.
- **Variance Modification**: Including natural distribution variance to avoid "flat average" imputation.
- **Global Median Fallback**: Used for extreme sparsity to prevent bias.

4.4 Frontend Team Process

[... Describe the development of the React application, UI/UX iterations, and integration with the backend API. ...]

4.5 Application of Methods and Theories

[Reflect on the engineering and analytical methods you applied (requirements analysis, architecture design, testing frameworks, etc.). Were the methods well-suited to the problem? Did you feel confident using them, or were there learning curves? What worked well and what fell short?]

4.6 Challenges During Execution

[Describe the most significant technical or organisational challenges encountered. How did the team respond? What was the outcome? Examples: "The I2C sensor driver took much longer than expected," or "The API contract changes caused significant rework in the frontend."]

4.7 Deviations from the Plan

[Compare what was planned versus what was actually executed. What changed and why? Were changes proactive (intentional pivots) or reactive (forced by circumstances)? How did the team adapt?]

4.8 Use of Tools and Technologies

[Reflect on the tools and technologies used throughout the project (IDEs, version control, project management, testing frameworks, etc.). Were they effective? Were there tools you wished you had used, or tools you regret choosing?]

5. PERSONAL REFLECTIONS

Piotr Junosz

Fourth semester work was so far the most challanging in terms of communication, expectations, performing and coordination within the big 11 people group. It was my first time trying to develop a solid project in such a big team consisting of 3 subteams, trying to make sense out of the important assigment.

The first significant change I noticed was how the motivation was working. It is well described by Victor Vroom's book "Work and motivation" where he says that effort is tied up to expectations (Vroom, 1964). So if you believe your hard work will result in a finished project, you are highly motivated, but if you look around and see that other people are not working, your expectation of success drops, and your motivation completely collapses. This was seen during this semester project period and led to periods of members not motivating themself as much as in previous semesters. Personally, this made the process of developing the system a lot less exciting.

Another correlated phenomenon that occured is called diffusion of responsibility where a person is less likely to take responsibility for an action when others are present (Darley & Latane, 1968). This has grown even more because the group had 11 people and it is easier to "hide" in this group than group consisting of 4-5 people. The larger a group becomes, the less individual work each member will take. This combined with problem of motivation made me realised few times that I have to work more as individual in order our project to succeed but then I understood that alone I cannot finish it fully which made me loose motivation and not enjoy the process of work on this project.

Having experienced those problems, I think what makes a big difference between a real job team and what i call "company simulation" of our big group is the role of a manager or a leader. Even though we were using Scrum roles such as Product owner and Scrum master, we were missing someone whos only responsibility would be focusing on connecting subteams work and generally leading the whole project process. I believe this could be a solution to at least the problem with motivation.

Damian Michal Choina

This project was my first time writing embedded systems code in C, and the learning curve at the start was real. Getting used to manual memory management, configuring hardware registers by hand, and

thinking about timing and interrupts in a way I never had to before took some adjustment. However, once the initial unfamiliarity wore off the work became genuinely enjoyable. I also had no previous experience setting up CI/CD at this scale. Building a GitHub Actions pipeline that automatically compiles the firmware, runs the full test suite, and uploads coverage reports on every pull request was something entirely new to me, and seeing it catch issues before they reached the hardware made the investment feel immediately worthwhile.

What made the technical challenges manageable was the sub-team dynamic. Because we already knew each other from previous SEP projects, we skipped the awkward early phase of figuring out how to work together and moved straight into productive collaboration. There was an existing level of trust that made it easy to ask for help, split work naturally, and review each other's code without it feeling like criticism. Communication with the ML and Frontend teams was also smooth throughout. We agreed on API contracts early and checked in regularly enough that integration never became a crisis.

Looking back, the things I would do differently are straightforward. I would get a minimal end-to-end path working in the first week rather than building depth in one area before the pieces connect. I would write tests as the code is written rather than treating them as something to catch up on later. And I would set up the CI pipeline at the very start of the project, not once the codebase is already growing. The automation overhead feels expensive early on but pays back quickly once the project reaches any real complexity.

Eduard Fekete

Learning Outcomes

This take may sound paradoxical but I believe now in some power of pointless meetings. I believed that if there is no agenda, then there is no reason to meet. I can see that both supervisor meetings with merely explaining what we have done would be great early on and also some team meetings when we would just talk about what we have done, alternatively with some team-building aspect could have had a large impact.

I also learned that information should be structured in a more hierarchical way. Strongest project-shaping decisions should always be visible in a simple format, more details can be linked to that and so on... I often referenced details and technicalities by saying "there is a file in the repo" or "you can see it on Figma", which in my head made sense, as I could fully orient in these environments but I could see towards the end that a large failure on my side as a Product Owner was that many people did not have a clear idea of what was where.

I also learned that there is a whole parallel universe going on besides school and work. It would be a nice utilitarianistic utopia to just let everyone deal with their own lives but it literally would improve collaboration to know what's going on with people besides the project. Sometimes I just feel like an hour of talking and 2 hours of work could have been more efficient than even 4 hours of working.

Contribution and Role

I was the Product Owner of the project, which meant that I was responsible for defining the product vision, managing the backlog, and ensuring that the team was aligned with our goals. I took this role seriously and tried to be as proactive as possible in communicating with the team and the supervisors. I tried to develop and communicate a strong product vision since the beginning, maintaining many of the diagrams and overall documentation. On top of that, I often ended up being "the person" for people who needed to quickly check on various decisions about to be made in the project, meaning I often found a lot of work even besides my typical work in my scrum team.

Although I believe that I have done my part in the project, I also feel that the way I approached everything could have been different. Different ways and channels of communicating all the information, better structuring of the information, and most importantly listening more to the team as often finding the best solution is not just about the problem itself but also about the team solving it.

I became a product owner mostly because this role stuck with me from previous projects and there was nobody else interested in it this time. A lot of trust and expectations were put on me, which I did my best to meet but I also can see that if I put more focus on the project, I could have achieved more. I remained my largest critic throughout the project, and I would love to have a peaceful mind concluding that I didn't get negative feedback, but how hypocritical would that be to say right after mentioning I didn't ask for enough feedback from the team?

Challenges and Growth

The most challenging aspect of the project was navigating the uncertainty of the situation overall. I can see many of my classmates, including myself, being affected by uncertainty and general confusion around the integration of artificial intelligence in learning and the industry in general. This affects motivation, and often prompts questioning of what is the point of our work and how should we approach getting help on such projects? How is it different talking with supervisors, searching around StackOverflow, asking for help in the team and asking AI?

Believing I could answer this well in a small reflection section in a random semester project would be ambitious and my confidence in solving world problems casually as part of my school curriculum is not that high. However, as I am expected to show growth in this section, and I am not planning on

uploading the history of my personal scale, I would summarize my way of overcoming this and dealing with the situation in a single quote - “talk with people”. It does not matter if this is about seeking supervision, lacking motivation to open the project rather than scrolling through social media or just generally feeling lost in the project. Almost always the answer is to talk with people, more is usually better.

Another challenge was the lack of a good structure and a highly related problem of a lack of transparency. The scrum teams were allowed to choose their own ways of working and they did. Frontend managed via Jira, IoT relied on Trello, MAL got immersed in Figma. On one hand - scrum utopia; every team choosing their own personal efficient way of working. On the other hand - a complete mess and a lack of transparency. The confusion about what is doing what, mostly across teams and the amount of accusations and paranoia that arised from that is unprecedented. I once completed a course about remote team leadership by Chris Croft, and among many of the takeaways was the importance of transparency and the feeling of inclusion in the project. What a mistake of not treating this project as a remote team. I wish I could have made everyone feel more included and more onboard about what is going on. Alignment meetings and a sea of DMs was not enough.

Yet another beautiful challenge I had to keep for the end was the problem with getting everyone onboard for specific technologies. I personally had my VPS with Coolify installed as a sort of “hammer for all nails” solution. I knew it might have not been the best but nobody else had better ideas available and I saw that even going with Azure, AWS, … would have a similar learning curve, this time with what seemed to be even less people knowing what is going on. I saved this challenge for the end to leave on a more positive note as I believe we have managed this well. People seemed to have researched about Coolify, everyone was given admin access since the beginning and every time I wasn’t available, various people decided to fix things on their own rather than to wait for me. And this was exactly what I wanted it to be. I am far from calling this perfection, but I am satisfied with what I could have influenced and what I have achieved on my side.

Reflection Forward

It is difficult to point out in several small chunks the key takeaways from such a project. On one hand, not having anything to point out suggests that a proper reflection has not been done, and on the other, allowing myself to summarize my experience into bullet points would be a massive oversimplification. However, these are my strongest points to be considered in future projects:

Being data-driven inwards, not just outwards - on the previous semester project I was able to work in a highly data-driven fashion and I considered it the peak of what can be achieved in terms of acting rationally on such projects. In the future, I will focus on not just scanning the business aspects of

features, and similar but also to understand better the team itself, skills, capabilities, obstacles, and so on.

Talk to people - I have very strongly outlined the idea behind this above but again, the best solution for uncertainty, and overall “human problems” is to talk to “humans”. Seeking supervision, understanding the situation beyond the project, aligning, and so on.

Being remote is not about the distance - Regardless of how inaccurate this point sounds, yes, sometimes even though we are in the same class, we communicate in highly asynchronous ways and as it is usual with Software projects, a lot is managed digitally. I will treat more projects in this way as disregarding the specific aspects of such projects can be problematic and the gap will reveal itself in nasty ways.

6. REFLECTION ON SUPERVISION

6.1 The Supervision Process

[Describe how supervision was conducted. How often did you meet? How did you prepare for meetings? What format did sessions typically take?]

6.1.1 MAL

Although we found brief moments of asking for advice in the beginning, around the elaboration phase we missed guidance that we could have sought. Towards the end, we set a goal of meeting every 1-2 weeks to never go off the path too much. Overall, we could have started earlier with asking for supervisor feedback and could have easily had less meetings towards the end if we had the correct trajectory from the start.

6.2 Impact on the Project

[In what specific ways did supervision change or improve the project? Were there suggestions or challenges from the supervisor that redirected your work? Were there times you disagreed with feedback — how did you handle that?]

6.2.1 MAL

The feedback radically changed and shaped the final approaches in the project. Not only did the advice mostly turn out to be true (predicting IoT data may be insufficient, mentioning which features could hinder or improve accuracy, etc.) but it also made us think about the problem in a more structured way. We had to justify our decisions and approaches to the supervisor, which forced us to be more critical of our own work and assumptions. This was a very valuable process that we could have benefited from even more if we had started it earlier.

6.3 Lessons for Future Projects

[What will you do differently in your next supervised project? Examples: prepare more structured agendas, present partial results earlier, ask more targeted questions, be more proactive about seeking feedback. Be concrete and actionable.]

As often, the main lesson is to be more proactive about seeking feedback and perhaps searching for efficient middle ground in many aspects. We correctly identified meetings of all to be inefficient and pointless, however, we suffered from alignment throughout the whole project. We sought supervision towards the end and missed it a lot in the middle. Overall, having a more structured start could have been of most advantage, it must, however, be noted that the semester project required utilizing the knowledge from other subjects which in the beginning made it difficult to have an accurate overview of the tasks to be accomplished and the approaches to be taken. We also had a lot of “silence and pointing fingers”, which is expected of large groups, where everyone expects someone else to bear the responsibility. Better defined roles and responsibilities as well as more structure and alignment could have fixed this problem and made the project more efficient and enjoyable.

7. CONCLUSION

7.1 Group Summary

[Summarise the group's overall experience. What defined this project's process? What are the most important things you collectively learned — about software engineering practice, teamwork, or the PBL learning model? How has this project prepared you for professional work?]

7.2 Recommendations

[Produce a practical list of do's and don'ts for future groups undertaking a similar project. These should be grounded in your actual experience — not generic advice.

Do:

- seek supervision early and often
- establish clear communication channels and regular check-ins
- define roles and responsibilities clearly from the start
- set up CI/CD pipelines early in the development process
- use tools to track progress and accountability

Avoid:

- leaving integration until the end
- neglecting testing until late in the project
- allowing motivation to drop without addressing it
- letting conflict fester without resolution
- assuming everyone is on the same page without regular alignment checks

8. REFERENCES

- Vroom, V. H. (1964). Work and motivation .
- Darley, J. M., & Latane, B. (1968). Bystander intervention in emergencies: Diffusion of responsibility. Journal of Personality and Social Psychology , 8 (4, Pt.1), 377–383. **https://doi.org/10.1037/h0025589**